ECE/CSC 547 – Cloud Architecture Project Proposal

Migrating a Biotech Startup's Local Data Preprocessing Pipeline to a Cloud-Native Architecture

Monish Erode Sridhar, Sanjaey Shunmuga Sundaram
MS in Computer Science, North Carolina State University

1. Introduction

1.1 Motivation

The idea for this project comes from a teammate's internship at a biotech startup, where the data preprocessing and annotation pipeline is used by multiple teams for analysis and visualization. The current setup works, but it has several practical problems because the pipeline is installed locally on each developer's or data scientist's machine. Updates, dependency issues, and inconsistent performance often slow down the workflow and make collaboration harder.

This project is motivated by the need to make that preprocessing system faster, easier to maintain, and more consistent by moving it to a managed cloud environment.

1.2 Executive Summary

The goal of this project is to redesign the company's data preprocessing pipeline as a **centralized, cloud-based service** instead of individual local installations. The plan is to containerize the existing Python pipeline and deploy it on **AWS** using either **AWS Batch** or **EKS**, so that the data stored in **S3** can be processed directly in the cloud.

This design would allow users to run preprocessing jobs through a single hosted system rather than their local machines. It would also make updates automatic, remove dependency conflicts, and keep logs and metrics in one place for easier monitoring. In short, the project aims to improve reliability, scalability, and maintainability of the company's data pipeline using cloud architecture principles.

2. Problem Description

2.1 The Problem

The company's data preprocessing and annotation pipeline currently runs on each developer's or data scientist's local machine. Every user must download raw assay data from AWS S3, run the preprocessing code locally, and then upload the processed results back to S3.

This setup causes several recurring issues:

- Manual data handling: Users repeatedly move large datasets between S3 and local storage, wasting time and bandwidth.
- Inconsistent performance: Machines have different hardware and processing power, resulting in variable runtimes and unpredictable throughput.
- Dependency conflicts: Local Python and OS versions often mismatch, leading to failed installations or incompatible runs.
- Manual updates: Every new version or patch must be manually installed, leading to version drift.
- Dispersed logging: Logs are stored on individual machines, making debugging and performance monitoring difficult.
- Tool compatibility issues: Several of the company's tools (e.g., visualization dashboards, ML analysis platforms) depend on this pipeline's outputs. Local inconsistencies can cause data mismatches or downstream failures.
- Limited cross-cloud connectivity: While most data resides on AWS, some of the company's services and analytics systems operate on GCP, creating barriers for data sharing and collaboration.

Overall, the current setup limits scalability, reliability, and integration across multiple cloud environments and user groups.

2.2 Business Requirements (BRs)

- **BR1 – Tenant Identification**
  Identify every user, team, or tool as a distinct tenant so that their jobs and data can be tracked independently.
- **BR2 – Availability**
  Keep the preprocessing system running with minimal downtime, even when some components fail.
- **BR3 – Hybrid-Cloud Interoperability**
  Allow controlled and secure data exchange between AWS (primary environment) and GCP (where some of the company's tools run).
- **BR4 – Metrics Collection**
  Collect quantitative metrics from the pipeline (for example, runtimes and error rates) to understand system health and performance.
- **BR5 – Log Collection**
  Collect and store logs from all components to support debugging, audits, and incident analysis.
- **BR6 – Consistency and Reliability**
  Ensure the same input data produces the same preprocessed output, regardless of tenant, workload, or cloud environment.
- **BR7 – Scalability and Elasticity**
  Handle time-varying workloads and allow multiple jobs and tenants to run in parallel without major performance degradation.
- **BR8 – Security and Compliance**
  Protect biological data and keep an audit trail of access to meet internal data-handling expectations.
- **BR9 – Cost Optimization**
  Monitor and manage resource usage so cloud spending stays under control across compute, storage, and data transfers.
- **BR10 – Tenant Isolation**
  Keep each tenant's data, configuration, and runtime environment separate to avoid cross-tenant access or interference.

- **BR11 – Tenant Authentication and Authorization**
  Verify tenant identity before access and restrict each tenant to only the resources and actions they are allowed to use.
- **BR12 – Tenant Usage and Billing Attribution**
  Record per-tenant resource usage so internal costs can be reviewed or allocated fairly.
- **BR13 – Ease of Maintenance**
  Make configuration, deployment, and rollback simple enough that the system can be maintained without heavy manual work.
- **BR14 – Backward Compatibility**
  Keep the company's existing tools working when the pipeline is updated or moved to the cloud.

2.3 Technical Requirements (TRs)

- ### BR1 – Tenant Identification

  TR1.1: Assign each tenant a unique ID or namespace.
  TR1.2: Maintain a registry that maps tenant IDs to specific users, teams, or tools.
  TR1.3: Log tenant creation, updates, and deletions.

  Necessity (BR1):
  TR1.1 defines the identity boundary.
  TR1.2 keeps tenant information consistent and discoverable.
  TR1.3 makes changes to tenant identities traceable over time.

  Sufficiency (BR1):
  Together, these are enough to uniquely identify tenants and track how tenant identities evolve.
  (WAF: Operational Excellence, Security)

- ### BR2 – Availability

  TR2.1: Add redundancy for critical compute and storage components to avoid single points of failure.
  TR2.2: Automatically retry jobs that fail due to transient issues.
  TR2.3: Define measurable uptime targets (for example, 99.9% monthly availability).
  TR2.4: Run continuous health and readiness checks on key components.

Necessity (BR2):
TR2.1 keeps the system running when one component fails.
TR2.2 prevents small network or instance glitches from stopping work.
TR2.3 makes “high availability” concrete and measurable.
TR2.4 detects failures early and directly depends on metrics and logs (BR3 and BR4).

Sufficiency (BR2):
These four cover prevention, detection, recovery, and verification of availability.
(WAF: Reliability)

- ## BR3 – Hybrid-Cloud Interoperability

TR3.1: Enable secure, authenticated data transfers between AWS and GCP.
TR3.2: Standardize on shared data formats and a simple, stable API contract for exchanged data.
TR3.3: Validate data integrity after cross-cloud transfers.
TR3.4: Monitor latency and throughput of cross-cloud transfers.

Necessity (BR3):
TR3.1 keeps cross-cloud movement safe.
TR3.2 avoids format mismatches and reduces integration friction.
TR3.3 ensures data is not corrupted in transit.
TR3.4 ensures that cross-cloud workflows are not too slow to be useful.

Sufficiency (BR3):
Together, they provide safe, consistent, and performant data exchange between AWS and GCP.
(WAF: Performance Efficiency, Operational Excellence)

- ## BR4 – Metrics Collection

TR4.1: Record metrics such as job runtime, queue wait time, throughput, and failure rate.
TR4.2: Store metrics in a way that allows aggregation and querying over time.
TR4.3: Tag metrics by tenant and job ID.

Necessity (BR4):
TR4.1 gives visibility into performance and reliability.
TR4.2 allows trends and patterns to be analyzed.
TR4.3 connects metrics to specific tenants and runs, which helps with debugging and fairness.

Sufficiency (BR4):
Together, these provide enough quantitative data to understand system health, support autoscaling (BR7), and verify availability targets (BR2).
(WAF: Reliability, Performance Efficiency)

- ## BR5 – Log Collection

TR5.1: Collect logs from all major components of the pipeline (ingest, preprocessing, output).
TR5.2: Centralize logs so they can be searched and correlated.
TR5.3: Include tenant ID and job ID in log entries where applicable.
TR5.4: Retain logs for a defined period to support audits and incident analysis.

Necessity (BR5):
TR5.1 ensures that important events are recorded.
TR5.2 makes investigation practical when issues occur.
TR5.3 links logs to specific tenants and jobs.
TR5.4 supports both debugging and compliance requirements.

Sufficiency (BR5):
These requirements provide enough logging to debug problems, support security investigations, and back up monitoring and availability.
(WAF: Operational Excellence, Security)

- ## BR6 – Consistency and Reliability

TR6.1: Record input data reference, code version, and configuration used for each run.
TR6.2: Validate outputs against expected schemas and simple statistical checks.
TR6.3: Use deterministic settings (for example, random seeds) wherever the pipeline has non-deterministic steps.

Necessity (BR6):
TR6.1 makes it possible to reproduce or explain differences in outputs.
TR6.2 prevents malformed or clearly incorrect outputs from propagating.
TR6.3 reduces run-to-run variation when the same input is processed.

Sufficiency (BR6):
These requirements are enough to support reproducible and reliable preprocessing, even as the system scales or moves across clouds.
(WAF: Reliability)

- ## BR7 – Scalability and Elasticity

  TR7.1: Adjust compute capacity based on the number of queued and running jobs.
  TR7.2: Allow multiple jobs from different tenants to run in parallel.
  TR7.3: Set per-tenant resource limits to prevent any single tenant from exhausting shared resources.

  Necessity (BR7):
  TR7.1 lets the system handle spikes and quiet periods.
  TR7.2 improves throughput and reduces waiting time.
  TR7.3 protects tenants from each other and avoids overload.

  Sufficiency (BR7):
  Together, these cover adapting to varying workloads while keeping the system stable for all tenants.
  (WAF: Performance Efficiency, Cost Optimization)

- ## BR8 – Security and Compliance

  TR8.1: Encrypt all data at rest and in transit.
  TR8.2: Log all data access and modifications.
  TR8.3: Enforce least-privilege access for all roles (tenants and admins).
  TR8.4: Retain security-relevant logs for a defined compliance period.

  Necessity (BR8):
  TR8.1 protects data confidentiality.
  TR8.2 and TR8.4 provide an audit trail for security incidents and reviews.
  TR8.3 limits the damage possible from any account.

  Sufficiency (BR8):
  These provide confidentiality, integrity, and accountability for the company's biological data.
  (WAF: Security)

- ## BR9 – Cost Optimization

  TR9.1: Track compute, storage, and data-transfer usage across tenants and workloads.
  TR9.2: Alert when spending or usage exceeds defined thresholds.
  TR9.3: Apply retention and lifecycle rules to remove or archive stale data and logs.

TR9.4: Consider both cost and performance when deciding where to run workloads (for example, AWS vs GCP).

Necessity (BR9):
TR9.1 gives visibility into where money is going.
TR9.2 helps detect cost problems early.
TR9.3 reduces long-term storage costs.
TR9.4 avoids overpaying for compute or data movement.

Sufficiency (BR9):
These are enough to keep compute, storage, and transfer costs under control while still meeting other BRs like availability and hybrid-cloud.
(WAF: Cost Optimization)

- ## BR10 – Tenant Isolation

TR10.1: Store each tenant's data in logically separate locations.
TR10.2: Restrict network communication paths between tenants.
TR10.3: Avoid sharing runtime resources that could expose one tenant's data to another.

Necessity (BR10):
TR10.1 prevents data from different tenants from being mixed.
TR10.2 blocks direct cross-tenant access.
TR10.3 reduces the risk of unintentional data leakage through shared components.

Sufficiency (BR10):
These three cover isolation at the data, network, and execution levels. Tenant Identification (BR1) underpins this by defining who each tenant is.
(WAF: Security)

- ## BR11 – Tenant Authentication and Authorization

TR11.1: Verify tenant identity before granting access.
TR11.2: Use role- or policy-based rules to limit what each tenant can do.
TR11.3: Re-validate credentials or tokens for long-running operations.

Necessity (BR11):
TR11.1 ensures only known tenants can use the system.
TR11.2 limits the actions they can perform.
TR11.3 keeps access valid and safe over time.

Sufficiency (BR11):
Together, these fully cover authentication and authorization for tenants.
(WAF: Security)

- ## BR12 – Tenant Usage and Billing Attribution

TR12.1: Record per-tenant usage of compute, storage, and data transfers.
TR12.2: Aggregate usage into simple internal cost reports.
TR12.3: Keep usage and cost history per tenant.

Necessity (BR12):
TR12.1 provides the raw usage data.
TR12.2 turns usage into interpretable cost views.
TR12.3 helps with long-term review and discussions with teams.

Sufficiency (BR12):
These are enough to support fair internal billing or cost-sharing discussions.
Tenant Identification (BR1) is a prerequisite to map usage to tenants.
(WAF: Cost Optimization, Operational Excellence)

- ## BR13 – Ease of Maintenance

TR13.1: Store configuration in version-controlled files.
TR13.2: Automate deployment, update, and rollback steps.
TR13.3: Document runbooks for common operational tasks and incident response.
TR13.4: Keep configuration separate from application code.

Necessity (BR13):
TR13.1 and TR13.4 make environments reproducible and easier to reason about.
TR13.2 reduces manual effort and errors during changes.
TR13.3 shortens time to recover when things go wrong.

Sufficiency (BR13):
These four requirements are enough to keep the system maintainable without heavy manual setup.
(WAF: Operational Excellence)

- ## BR14 – Backward Compatibility

TR14.1: Version outputs and APIs, and publish deprecation timelines.
TR14.2: Validate that new outputs still match existing schemas used by current

tools.
TR14.3: Provide temporary compatibility layers for at least one release cycle.
TR14.4: Track which tools depend on which versions of outputs or APIs.

Necessity (BR14):
TR14.1 controls how changes are introduced.
TR14.2 catches breaking changes before they reach users.
TR14.3 gives teams time to migrate.
TR14.4 helps plan deprecations and support.

Sufficiency (BR14):
Together, these ensure the company's existing tools keep working while the pipeline moves to a cloud-native design and evolves over time.
(WAF: Reliability)

2.4 Trade-offs and Conflicting Requirements

Conflict 1: BR2 (Availability) vs BR7 (Scalability and Elasticity) Improving availability often requires more health checks, retries, and redundant capacity, while scalability focuses on dynamic resource changes driven by workload spikes. When the system scales aggressively, it can temporarily reduce the redundancy or spare capacity that availability relies on. At the same time, availability mechanisms like frequent retries can increase load during peak scaling periods. These two objectives can push the system in different directions and must be balanced carefully.

Conflict 2: BR3 (Hybrid-Cloud Interoperability) vs BR8 (Security and Compliance) Sharing data across AWS and GCP increases flexibility for teams that rely on tools in different clouds, but it also expands the attack surface. Each additional cross-cloud link requires more identity checks and more potential points of failure. Strong isolation and strict access boundaries can sometimes limit how easily the two environments can communicate. This creates a direct tension between interoperability and security.

Conflict 3: BR7 (Scalability and Elasticity) vs BR6 (Consistency and Reliability) Running many preprocessing jobs in parallel helps throughput, but it also increases the risk of nondeterministic results, especially when jobs run across different nodes or behind different performance profiles. High concurrency can make it harder to guarantee strict consistency and reproducible outputs. These goals naturally compete when workloads scale quickly.

3. Provider Selection

3.1 Criteria for Choosing a Provider

To select a suitable cloud provider, we compared the three major platforms: **AWS**, **Google Cloud Platform (GCP)**, and **Microsoft Azure**, based on how well they meet the requirements listed in Section 2.3.

To support the BRs and TRs from Section 2, we need a cloud provider that can:

1. **Support multi-tenant identity and access control**
   Fine-grained IAM-style roles and policies for tenants and internal teams (BR1, BR10, BR8).

2. **Provide reliable compute for batch/Job-style workloads**
   Ability to run containerized preprocessing jobs at scale with retries and health checks (BR2, BR6, BR7).

3. **Offer strong observability tools**
   Built-in metrics, logging, and alerting that integrate with our services and support per-tenant tagging (BR3, BR4).

4. **Integrate well with object storage and data pipelines**
   Native support for large-scale object storage, versioning, lifecycle rules, and event-driven processing (BR5, BR6, BR9).

5. **Support hybrid-cloud or multi-cloud workflows**
   Secure, authenticated transfer paths to/from GCP, plus reasonable control over network and routing (BR3, BR5).

6. **Provide solid security and compliance features**
   Encryption, key management, audit logs, and network isolation primitives (BR8, BR10).

7. **Offer cost visibility and basic cost-control tools**
   Per-service and per-tag/tenant cost breakdown, budgets, and alerts (BR9, BR12).

8. **Have good documentation and ecosystem fit**
   Strong SDKs and libraries in Python, good community support, and services that match common patterns for batch data processing.

These criteria are driven directly by our BRs/TRs: identity and isolation for multi-tenancy, batch-style compute, observability, hybrid-cloud needs, security, and cost optimization.

Each provider was scored on these criteria using a 1-to-5 scale

3.2 Provider Comparison

Criterion	AWS	Azure	GCP	Justification (3 bullets)
Multi-tenant IAM / Access Control	**5**	**4**	**4**	1. **AWS (5):** IAM is highly granular with mature policy-based access and resource-level permissions. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html 2. **Azure (4):** Strong RBAC model, but policy depth and resource-level granularity are somewhat less flexible than AWS. https://learn.microsoft.com/en-us/azure/role-based-access-control/overview 3. **GCP (4):** Good IAM system but fewer fine-grained permissions across all services compared to AWS. https://cloud.google.com/iam/docs
Batch / Container Compute	**5**	**4**	**4**	1. **AWS (5):** Multiple mature options for batch workloads (e.g., Batch, ECS, EKS), widely used for pipeline-style jobs. https://aws.amazon.com/batch 2. **Azure (4):** Azure Batch and Container Apps are strong offerings, but would require more integration effort with the company's AWS-based data. https://learn.microsoft.com/en-us/azure/batch/batch-technical-overview 3. **GCP (4):** Cloud Run and GKE support containers well, but less aligned with our existing AWS-heavy workflows. https://cloud.google.com/run
Metrics / Monitoring	**5**	**4**	**4**	1. **AWS (5):** Very strong integrated metrics/monitoring with CloudWatch; fits TRs tied to availability and observability. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.htm 2. I **Azure (4):** Azure Monitor is powerful but

				less naturally tied into the company's existing AWS systems. https://learn.microsoft.com/en-us/azure/azure-monitor/overview 3. **GCP (4):** Cloud Monitoring is solid, but cross-cloud visibility with AWS is weaker. https://cloud.google.com/monitoring
Centralized Logging / Audit Trails	**5**	**4**	**4**	1. **AWS (5):** CloudTrail and unified log storage support detailed audit tracking, which helps satisfy BR4/BR8. https://docs.aws.amazon.com/cloudtrail 2. **Azure (4):** Log Analytics and Monitor work well but require heavier cross-cloud setup. https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-overview 3. **GCP (4):** Good logging, but multi-cloud correlation is not as mature as AWS's ecosystem. https://cloud.google.com/logging
Object Storage + Lifecycle Management	**5**	**4**	**4**	1. **AWS (5):** S3 offers excellent durability, lifecycle rules, and versioning - core to our preprocessing pipeline. https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html 2. **Azure (4):** Blob Storage is capable but migration from S3 would add overhead. https://learn.microsoft.com/en-us/azure/storage/blobs/storage-lifecycle-management-concepts 3. **GCP (4):** GCS is strong but does not match the company's existing S3-based ecosystem. https://cloud.google.com/storage/docs/lifecycle
Hybrid / Multi-cloud Support	**4**	**3**	**4**	1. **AWS (4):** Supports hybrid architectures but AWS↔GCP requires custom setups. https://aws.amazon.com/hybrid 2. **Azure (3):** Focuses mainly on Azure–on-

				prem hybrid; not ideal for AWS↔GCP interoperability. https://learn.microsoft.com/en-us/azure/architecture/hybrid/hybrid-overview 3. **GCP (4):** Good hybrid features; similar limitations as AWS for AWS↔GCP direct connectivity. https://cloud.google.com/hybrid
Security (Encryption, Isolation)	**5**	**4**	**4**	1. **AWS (5):** Strong encryption, KMS, IAM, and VPC isolation - very mature ecosystem. https://docs.aws.amazon.com/kms 2. **Azure (4):** Solid security stack but requires more integration effort with AWS-based data. https://learn.microsoft.com/en-us/azure/security/fundamentals/overview 3. **GCP (4):** Good security features but isolation options are less granular than AWS. https://cloud.google.com/security
Cost Visibility / Tagging / Budgets	**5**	**4**	**4**	1. **AWS (5):** Excellent cost breakdown per service, per tag, and per tenant - directly helps BR9/BR12. https://docs.aws.amazon.com/cost-management **Azure (4):** Good cost tools but harder to use in a multi-cloud AWS↔GCP setup. https://learn.microsoft.com/en-us/azure/cost-management-billing/cost-management-billing-overview **GCP (4):** Good cost reporting but tagging is less consistent across services. https://cloud.google.com/billing/docs
Fit for Company Workloads	**5**	**3**	**3**	1. **AWS (5):** The company's entire data storage, pipelines, and tools already depend on S3 and AWS workflows.

				2. **Azure (3):** No existing company components run here → highest integration cost. 3. **GCP (3):** Only specific analytics tools run on GCP, not the core data path.

Total Scores:

Provider	Total Score
AWS	**44**
Azure	**34**
GCP	**35**

Winner:
Highest total score, existing company workflows already on AWS, strongest fit for identity, logging, metrics, batch compute, and S3-based pipeline.

3.3 Final Selection

Based on the comparison in Section 3.2, **AWS** is the provider that best satisfies our TRs and aligns with the company's existing environment. AWS received the highest total score (44), mainly because:

- The company's raw and processed data already lives in **Amazon S3**, so AWS avoids migration overhead.
- AWS offers mature identity and multi-tenant access controls, which are important for **BR1 (Tenant Identification)** and **BR10 (Tenant Isolation)**.
- The pipeline relies heavily on batch-style workloads, and AWS has strong options for containerized job execution.
- Observability (metrics + logs) is central to **availability** and **scalability**, and AWS has the most complete monitoring/logging ecosystem for our needs.
- AWS also provides better cost visibility and per-tenant tagging than the other providers, which supports **BR9 (Cost Optimization)** and **BR12 (Tenant Usage and Billing Attribution)**.

Although GCP is required for hybrid use cases (because some of the company's tools run there), it does not offer compelling advantages as the *primary* environment. Azure ranked lowest because none of the company's workloads currently run on Azure, so adopting it would add unnecessary overhead.

Final choice: AWS as the primary environment, with GCP as a secondary cloud for interoperability.

3.3.1 List of Services from AWS (Winner)

Below is the set of AWS services relevant to our TRs.

Identity & Access

- AWS Identity and Access Management (IAM)
  https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

Compute / Job Execution

- AWS Batch
- Amazon ECS / EKS (container orchestration)
  https://aws.amazon.com/batch
  https://aws.amazon.com/ecs
  https://aws.amazon.com/eks

Storage

- Amazon S3 (object storage, lifecycle rules, versioning)
  https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

Metrics / Monitoring

- Amazon CloudWatch (metrics, dashboards, alarms)
  https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

Logging / Auditing

- AWS CloudTrail (audit logs)
- CloudWatch Logs (application logs)
  https://docs.aws.amazon.com/cloudtrail
  https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html

Networking / Hybrid Integration

- AWS VPC, VPN, Direct Connect (for cross-cloud connections)
  https://aws.amazon.com/directconnect

Key Management / Security

- AWS Key Management Service (KMS)
  https://docs.aws.amazon.com/kms

Cost Management

- AWS Cost Explorer, Billing Dashboard
  https://docs.aws.amazon.com/cost-management

Detailed Description of Two Services per Team Member

Service 1 (Monish): AWS Identity and Access Management (IAM)

IAM lets us define **users, roles, and policies** that control exactly which resources a tenant can access. For our multi-tenant pipeline, IAM is essential because:

- It lets us represent **each tenant as a separate identity** (BR1).
- We can restrict access so tenants only see their own datasets, queues, or job triggers (BR10, BR11).

- IAM policies support tags, which helps with tenant-level auditing and cost tracking.
- It creates clear isolation boundaries, which also supports BR8 (Security) and BR12 (Usage Attribution).

IAM is foundational for making the pipeline safe for multiple teams running jobs simultaneously.

Service 2 (Monish): Amazon CloudWatch (Metrics and Alerts)

CloudWatch is AWS's metrics and monitoring system. It helps us collect:

- job runtimes
- error rates
- queue lengths
- throughput
- custom pipeline metrics

This directly supports BR3 (Metrics Collection) and helps maintain BR2 (Availability), because outages can't be detected without metrics. CloudWatch also supports alerts, which connect to BR4 (Log Collection) and BR7 (Scalability/Elasticity), since autoscaling often depends on workload metrics.

CloudWatch basically gives us visibility into pipeline behavior so we can maintain performance and reliability.

Service 3 (Sanjaey): Amazon S3 (Storage + Lifecycle Rules)

Amazon S3 is where the company already stores all raw assay data and processed outputs. It supports:

- versioning
- lifecycle rules (archive/delete)
- strong durability
- per-object metadata
- bucket policies for tenant-level isolation

S3 helps satisfy multiple BRs:

- BR5 (Hybrid Cloud Interoperability) - S3 is the AWS side of all cross-cloud transfers
- BR6 (Consistency & Reliability) - versioning helps track input/output pairs
- BR9 (Cost Optimization) - lifecycle policies help control storage cost

Since the preprocessing pipeline reads and writes all its data through S3, this service is central to the project.

Service 4 (Sanjaey): AWS CloudTrail (Audit Logging)

CloudTrail records **all API calls**, including data access, job triggers, bucket reads/writes, and IAM actions. This is required for:

- BR4 (Log Collection) - we need logs across all services
- BR8 (Security & Compliance) - we need to prove who accessed what
- BR10 (Isolation) - audit logs help detect cross-tenant access violations
- BR12 (Billing Attribution) - access logs support usage tracking

CloudTrail gives us the long-term audit record needed for both security and troubleshooting.

4. First Design Draft

4.1 Basic Building Blocks of the Design

At this stage, the main building blocks we are considering come directly from the TRs and the capability survey in Section 3. The idea is to combine these services in a way that supports multi-tenant preprocessing, observability, and hybrid-cloud data movement. The key pieces are:

Identity & Tenant Isolation

- **AWS IAM** for tenant roles, access boundaries, and identity separation.

Compute Layer for Preprocessing Jobs

- A managed **container-based compute** service (e.g., a batch-style compute runner) to run preprocessing tasks on demand.
- Support for retries, health checks, and scaling to match workload spikes.

Storage & Data Management

- **Amazon S3** for storing raw assay data and processed outputs.
- Lifecycle rules, versioning, and object-level tagging for tenant-level tracking.

Observability Layer

- **CloudWatch metrics** to collect job runtimes, error counts, and workload indicators.
- **CloudTrail + centralized logs** to track data access, debugging information, and audit history.

Hybrid/Inter-cloud Connectivity

- **Secure transfer paths** between AWS and GCP for dashboards/tools that need processed data.

Security and Compliance Controls

- **KMS-managed encryption** for both data at rest and in transit.
- IAM policies and log retention for auditability.

Cost Tracking

- **Cost Explorer + tagging** for per-tenant billing breakdowns.

These are not design decisions yet; they are just the main elements we expect to combine into the architecture.

4.2 Top-Level, Informal Validation of the Design

This section is meant to be “hand-wavy,” so the arguments don’t need to be rigorous. The goal is simply to show that the building blocks we listed are capable of satisfying the high-level TRs.

Why the design should work (informally):

- **Tenant Identification & Isolation (BR1, BR10, BR11):**
  IAM roles, S3 prefixes/tags, and compute-level permissions should be enough to keep each tenant’s data and jobs separate. As long as each tenant has its own identity and restricted policies, we can enforce boundaries without requiring programming changes.
- **Availability (BR2):**
  Using a managed batch-style compute service plus redundant storage should help avoid downtime. Automatic retries and health checks are built into AWS services, so the system should be able to continue running even if some nodes fail.
- **Metrics + Logs (BR3, BR4):**
  CloudWatch and CloudTrail already provide metrics, logs, and auditing across all AWS components. Since preprocessing jobs run in containers, we can push custom metrics and logs as well. This should give enough visibility to detect issues early.
- **Hybrid-Cloud Interoperability (BR3):**
  Storing all data in S3 first and then transferring results to GCP through authenticated requests should be workable. Many teams already do this in practice, so this approach is realistic.

- **Consistency and Reliability (BR6):**
  Versioning input data, output data, and configuration files should give us reproducible runs. As long as the compute environment is containerized, the processing behavior should be stable.
- **Scalability and Elasticity (BR7):**
  Container-based compute platforms can scale horizontally, so running multiple jobs in parallel across tenants should be feasible. CloudWatch metrics should help autoscaling react to load.
- **Security and Compliance (BR8):**
  Encryption (KMS), logging, IAM boundaries, and retention rules should cover confidentiality and audit needs. This aligns with how cloud platforms normally handle sensitive workloads.
- **Cost Optimization (BR9) + Tenant Billing (BR12):**
  AWS supports detailed cost tracking through tagging and cost reports. Since each tenant can be tagged, cost attribution should be straightforward.
- **Backward Compatibility (BR14):**
  S3 versioning and API versioning should allow older tools to continue working even if we change the preprocessing pipeline in the cloud.

5. Second Design Draft

5.1 Use of the Well-Architected Framework

The Well-Architected Framework suggests a structured way to move from requirements to an actual design. Based on Section 4.3 of the class notes, the main steps we followed are:

1. **Identify the workload and main objectives**
   In our case, the workload is the multi-tenant preprocessing pipeline, and the objectives come from the BRs in Section 2.2.
2. **Translate the objectives into technical requirements (TRs)**
   We refined the BRs into TRs that can be evaluated by cloud services.
3. **Review the workload using the five AWS pillars**
   We focused on Reliability, Security, and Performance (professor-required), and added Cost Optimization as the extra pillar.

4. **Find risks or weak areas in each pillar**
   For example, in the Reliability pillar, a weak area was the lack of retries and metrics for scaling.
5. **Select services and design choices that reduce those risks**
   IAM for tenant boundaries, S3 versioning for consistency, metric-driven compute scaling, etc.
6. **Document decisions and prepare for tradeoff discussions**
   We identified conflicting BRs/TRs (e.g., availability vs cost) and addressed them in Section 5.6.

These steps guided how we shaped the design, even though the final detailed architecture will appear in later sections.

5.2 Discussion of Pillars

Pillar 1 - Reliability (Monish)

The Reliability pillar focuses on keeping the system running correctly even when parts fail. The AWS document and our class notes highlight a few key ideas: automatic recovery, test-ready failover paths, and monitoring that drives response actions.

For our pipeline, the main reliability concerns are job failures, node failures, and inconsistent tenant performance. The design reflects the pillar in these ways:

- We rely on **managed compute** with built-in retries, which reduces the chance that a temporary node issue causes a full job failure.
- We use **metrics** (queue length, job error rate) to detect backpressure early.
- **S3 versioning** helps ensure reproducible results even if a job needs to be re-run.
- **Identity separation** also helps reliability because tenants cannot interfere with each other's workloads.

These align with the Reliability pillar's guidance to "monitor, test, and automatically recover" whenever possible.

Pillar 2 - Security (Sanjaey)

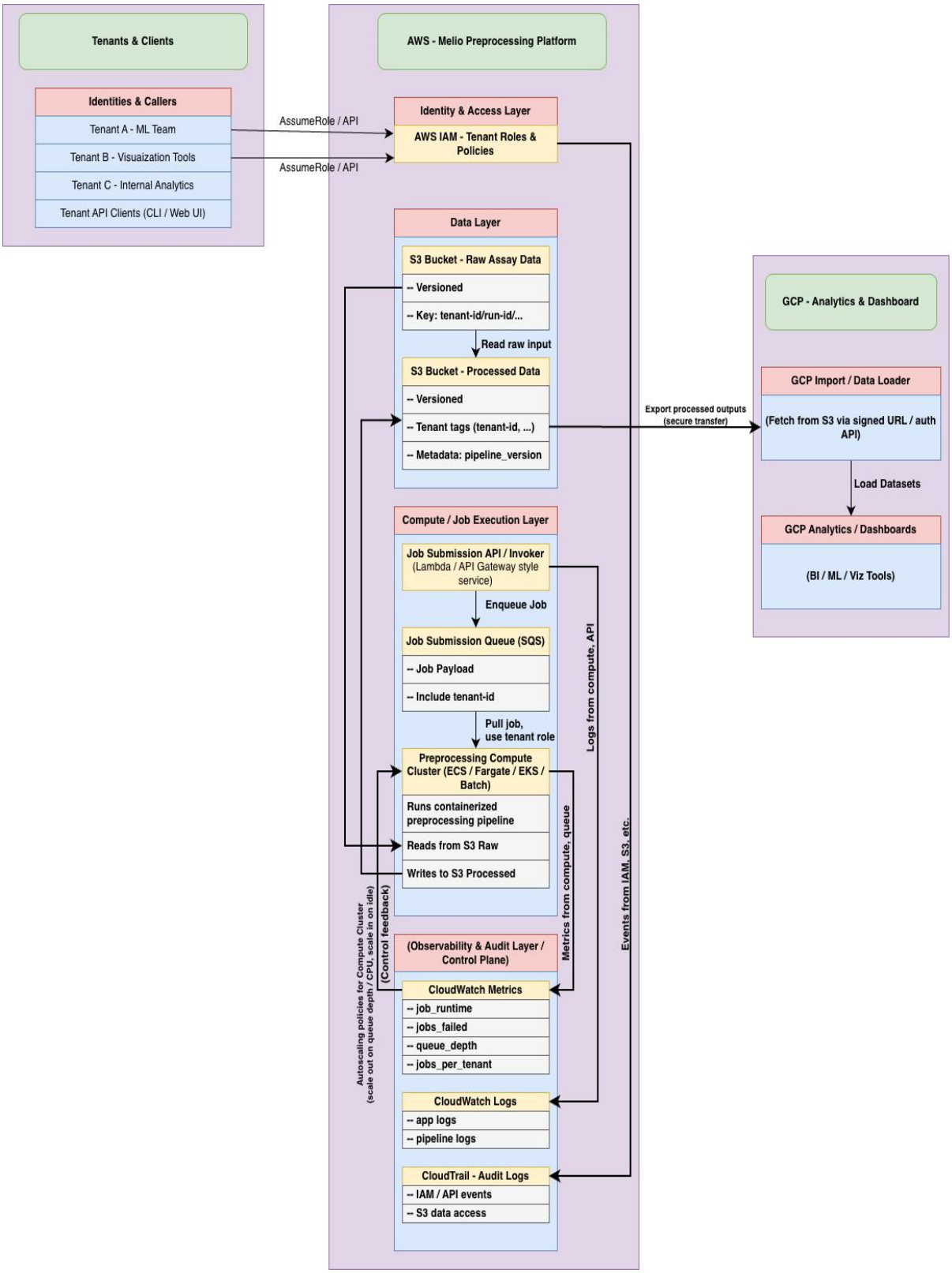
The Security pillar emphasizes identity, data protection, logging, and enforcing least privilege. Since the pipeline processes biological assay data, this pillar is very important.

The design reflects the Security pillar in these ways:

- **IAM identities per tenant** enforce strong isolation and prevent cross-tenant access.
- **KMS-based encryption** protects data both in S3 and during transfers.
- **CloudTrail logs** ensure every data access event is recorded for auditing.
- **Segmentation of dev/test/prod** reduces accidental exposure and keeps experiments separate from real samples.

This follows the Security pillar's general advice: "strong identity, strong boundaries, and strong visibility."

5.3 Use of CloudFormation Diagrams



The diagram above shows the top-level architecture derived from the CloudFormation resources listed in Section 5.1. The design is organized into three columns - Tenants, the AWS preprocessing platform, and the GCP analytics environment - and the AWS column is further separated into four layers: Identity & Access, Data, Compute/Job Execution, and the Control/Observability Plane. This structure follows the layout used in the class notes.

Each arrow in the diagram represents a clear interaction or flow between components, such as job submission, S3 reads and writes, secure data export, and the collection of metrics, logs, and audit events. IAM sits at the entry point to enforce tenant isolation, the S3 buckets form the data layer, the queue and compute cluster handle execution, and CloudWatch/CloudTrail record operational visibility.

The GCP side only receives processed outputs through a controlled transfer path. Overall, the diagram connects the CloudFormation building blocks and shows how data, identity, and observability fit together in the proposed architecture.

5.4 Validation of the Design

Here we give more concrete arguments for how the design satisfies the TRs:

Tenant Identification (TR1.*)

IAM roles and S3 prefix policies directly enforce tenant boundaries. Each tenant can only assume its own role, and roles restrict access to that tenant’s buckets and job queues. Because CloudTrail logs IAM events, we also meet TR1.3 (usage tracking).

Availability (TR2.*)

The compute layer uses managed containers with automatic retries and health checks. These services restart failing jobs without manual intervention. S3 provides eleven-nines durability, which helps ensure input/output persistence. This fulfills redundancy and recovery TRs.

Metrics & Logs (TR3.* & TR4.*)

CloudWatch receives metrics from the compute tasks (CPU, memory, queue length, job failures). These are the same metrics needed for autoscaling (TR6.1), so elasticity is driven by meaningful data. CloudTrail + CloudWatch Logs centralize audit trails and operational logs, satisfying TR4.x.

Hybrid Cloud (TR5.*)

Data movement occurs only after storage in S3. Using signed URLs or authenticated requests satisfies the secure-transfer TR. S3 versioning ensures file integrity, supporting TR5.3.

Consistency & Reliability (TR6.*)

Reproducibility is achieved through containerized execution and S3 versioning. These ensure the same input and code version lead to the same output. Validation checks enforce schema correctness.

Scalability (TR7.*)

Compute scaling uses CloudWatch metrics, such as queue depth or CPU saturation. This satisfies TR7.1 (metric-driven scaling). Multi-tenant workloads run in parallel because tasks run independently in isolated containers (TR7.2).

Security (TR8.*)

IAM + KMS + CloudTrail cover identity, encryption, and auditing. Strict least-privilege policies and environment segmentation match all TR8.x requirements.

Cost Optimization (TR9.*)

All resources can be tagged by tenant, which feeds directly into Cost Explorer. S3 lifecycle rules satisfy TR9.2 for long-term storage reduction.

Maintenance (TR13.*)

Containerization, versioned config files, and centralized logs support quicker rollout and debugging.

Overall, the design meets the TRs from at least four WAF pillars: Reliability, Security, Performance, and Cost.

5.5 Design Principles and Best Practices Used

The class notes list a number of design principles (Topic 04, pages 22–27) . Here are the ones we used:

1. Favor managed services

We rely on managed batch/compute, S3, IAM, and CloudWatch. This reduces operational overhead and increases reliability.

2. Design for automatic recovery

Managed compute restarts tasks automatically. Logging and metrics help detect and respond to failures.

3. Stop guessing capacity

Compute scaling uses real metrics (queue length, CPU). We do not set fixed capacity.

4. Infrastructure as Code

CloudFormation templates help us deploy consistently and avoid manual configuration drift.

5. Enforce least privilege

IAM roles and policies are scoped to each tenant, respecting the Security pillar guidance.

6. Version everything

Versioned buckets and container images help control consistency and reproducibility.

5.6 Tradeoffs Revisited

Tradeoff 1 (Monish): Reliability vs Performance Efficiency

Highly reliable systems often introduce extra redundancy, retries, and durability mechanisms. These techniques improve fault tolerance but can reduce performance efficiency, especially for batch workloads.

Using the “even swaps” method:

- **Objective A: Improve reliability**
  Techniques: retries, health checks, durable queues, versioned storage.
- **Objective B: Maintain performance efficiency**
  Techniques: minimal overhead, low startup latency, fewer checkpoints.
- **Swap reasoning:**
  Options involving aggressive checkpointing were eliminated because they slowed down preprocessing jobs significantly. Less frequent checkpoints and retry-based recovery preserved reliability while keeping throughput reasonable.
- **Final stance:**
  We kept redundancy and retries (important for TR2.x) but avoided heavy durability overhead that would slow down job execution.

Tradeoff 2 (Sanjaey): Hybrid Cloud Interoperability vs Security

Allowing data to move to GCP helps teams who rely on GCP-based dashboards but increases the security surface area.

Even swaps analysis:

- **Benefit:** GCP teams get immediate access to processed datasets.
- **Risk:** More network paths and authentication mechanisms to secure.
- **Swap reasoning:** The risky option (direct cross-cloud networking) was eliminated. Instead, transfer only through signed URLs or controlled API endpoints.
- **Final stance:** Accept interoperability but with *tight* security boundaries and minimal points of contact.

These two tradeoffs capture the major tensions in the design: balancing uptime vs. cost and flexibility vs. security.

6 Kubernetes Experimentation

This section describes the Kubernetes experiments we used to validate that the autoscaling and self-healing mechanisms in our architecture behave as expected. In our project, the Kubernetes deployment is a simplified stand-in for the company's cloud-hosted preprocessing workers: each pod runs a small Flask service that exposes CPU-intensive endpoints and is scaled by a Horizontal Pod Autoscaler (HPA).

This experiment validates the following Technical Requirements from Section 2.3:

- **TR7.1 – Adjust compute capacity based on workload intensity.**
- **TR7.2 – Allow multiple jobs from different tenants to run in parallel.**
- **TR2.1 – Add redundancy so the system continues operating when a worker fails.**
- **TR2.4 – Continuously run health/readiness checks and remediate failures automatically.**
- **TR4.1 – Record metrics that support autoscaling and monitoring.**

Together, these TRs capture the reliability and elasticity properties that the final AWS-based design must provide for the company's preprocessing pipeline.

6.1 Experiment Design

6.1.1 Environment and Kubernetes Configuration

The experiments were run on a local two-node Minikube cluster (docker driver) on macOS, using the Kubernetes manifests in the k8s/ folder.

Cluster configuration

- Minikube profile: k8-project
- Nodes: 1 control-plane + 1 worker node
- Kubernetes version: v1.34.0

Application components

- **Deployment** (deployment.yaml)
    - Name: service
    - Initial replicas: 3 (overridden dynamically by the HPA)
    - Label: app: payment (used by the Service and HPA)
    - Container image: sanjaey/server:latest
    - Container port: 8000
    - Resource configuration: requests cpu: 100m, limits cpu: 500m
    - Liveness probe: HTTP GET on /health every 5 seconds. Pods failing the probe are restarted automatically.
- **Service** (service.yaml)

    - Name: payment-service
    - Type: NodePort
    - Selects pods with label app: payment
    - Exposes port 8000 inside the cluster, mapped to NodePort 30007 on each node.
- **Horizontal Pod Autoscaler** (hpa.yaml)
    - Name: payment-hpa
    - Target: Deployment service
    - minReplicas: 1, maxReplicas: 10
    - Metric: CPU utilization with averageUtilization: 50

This setup mirrors the final AWS design at a small scale: the Deployment represents the pool of preprocessing workers, the Service is the internal entry point for jobs, and the HPA plays the role of metric-driven autoscaling logic.

6.1.2 Application Behavior

The Flask application (server.py) is intentionally simple and CPU-bound:

- GET /health – Returns 200 OK and is used by both Kubernetes (liveness probe) and Locust (client-side checks) to confirm that a pod is alive.

- GET /load – Executes a tight loop to burn CPU for a short period and then returns a response. This endpoint is a proxy for the CPU-intensive preprocessing step in the real pipeline.

Because /load performs busy waiting with no external I/O, it is ideal for stressing the CPU and triggering the HPA when many concurrent requests are issued.

6.1.3 Inputs and Expected Outputs

We follow a three-phase experiment protocol that produces both quantitative outputs (metrics and pod counts) and qualitative outputs (screenshots of kubectl and Locust).

Phase 1 – Baseline (Idle Cluster)

1. Deploy the manifests and wait until the cluster stabilizes: kubectl get hpa payment-hpa and kubectl get pods -l app=payment.
2. With no external traffic, the HPA scales the Deployment down to minReplicas = 1. CPU utilization remains around 1–3%.

Expected outputs:

- Exactly one service-... pod is in Running state.
- The HPA reports TARGETS close to cpu: 1–3%/50% and REPLICAS = 1.

Phase 2 – Load and Autoscaling (TR7.1, TR7.2)

1. Expose the Service locally using port-forwarding: kubectl port-forward svc/payment-service 8000:8000.
2. Use Locust to generate load, targeting primarily the /load endpoint (Section 6.2).
3. Monitor the autoscaler and the pods concurrently:
    - kubectl get hpa payment-hpa -w
    - kubectl get pods -l app=payment -w

Expected outputs:

- As the number of simulated users increases, CPU utilization rises significantly above the 50% target (up to ~289%).
- The HPA increases the replica count from 1 to at least 4–6 pods, visible in the REPLICAS column and the creation of new pods.
- New pods transition through Pending → ContainerCreating → Running while existing pods stay healthy.
- Locust continues to report successful requests with acceptable latencies, showing that traffic is distributed across multiple pods.

Phase 3 – Self-healing under pod failure (TR2.1, TR2.4)

1. Once the system is stable with several replicas, delete one of the pods manually, e.g.: kubectl delete pod service-578d4445b5-bdxk4.
2. Keep kubectl get pods -l app=payment -w running to observe the reaction.

Expected outputs:

- The deleted pod enters the Terminating state.
- A new pod with a different name appears almost immediately in Pending and transitions to Running.
- At all times, multiple service-... pods remain Running 1/1, so the Service continues to serve traffic without interruption.

Although TR7.1 refers to scaling based on queued or running jobs, in this Minikube experiment the HPA scales based on CPU utilization instead. This is acceptable for validating TR7.1, because the goal is to show that the system can automatically adjust compute capacity in response to a workload metric. In the full AWS design, this scaling signal would come from an actual job queue.

6.2 Workload Generation with Locust

Locust is used as the workload generator because it can simulate concurrent clients and provides convenient latency statistics.

We launch Locust from the project root:

```
locust -f locustfile.py
```

and configure the test using the web UI.

User behavior

The script (locustfile.py**)** defines a single user class:

```
from locust import HttpUser, task, between

class PaymentServiceUser(HttpUser):
    # Think time between consecutive requests from each simulated user
    wait_time = between(0.1, 0.5)

    @task(9)
    def hit_cpu_load(self):
        # Main endpoint to trigger CPU utilization and HPA scaling
        self.client.get("/load")

    @task(1)
    def health_check(self):
        # Occasionally call /health to verify that pods are alive
        self.client.get("/health")
```

Each Locust user:

- Sends 90% of its requests to /load, which simulates CPU-heavy preprocessing work. Sends 10% of its requests to /health, which acts like a lightweight client-side liveness check.
- Waits a short time between requests, ensuring a steady, high-volume stream of load.

Test configuration

In the Locust UI we configure:

- **Number of users:** 10
- **Spawn rate:** 1 user/second
- **Host:** http://localhost:8000

This configuration is shown in *Figure 6.1*.

After the test starts, the Statistics tab (Figure 6.2) reports the number of requests, failure counts, and latency metrics (median, 95th percentile, 99th percentile) for /load and /health. For our runs,

both endpoints had single-digit millisecond latencies at small scale, indicating that a single pod can handle low concurrency easily. The interesting behavior comes from how Kubernetes reacts when many users execute /load concurrently and CPU utilization surges.

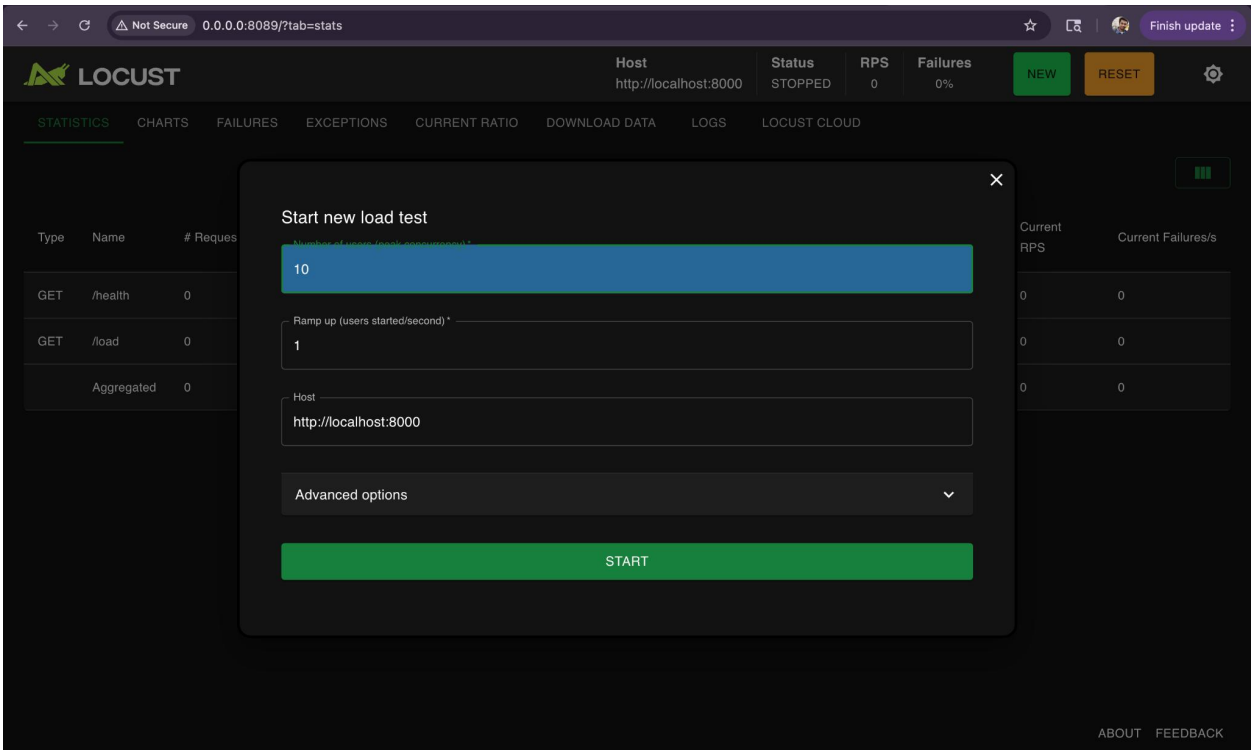


Figure 6.1: Locust “Start new load test” dialog

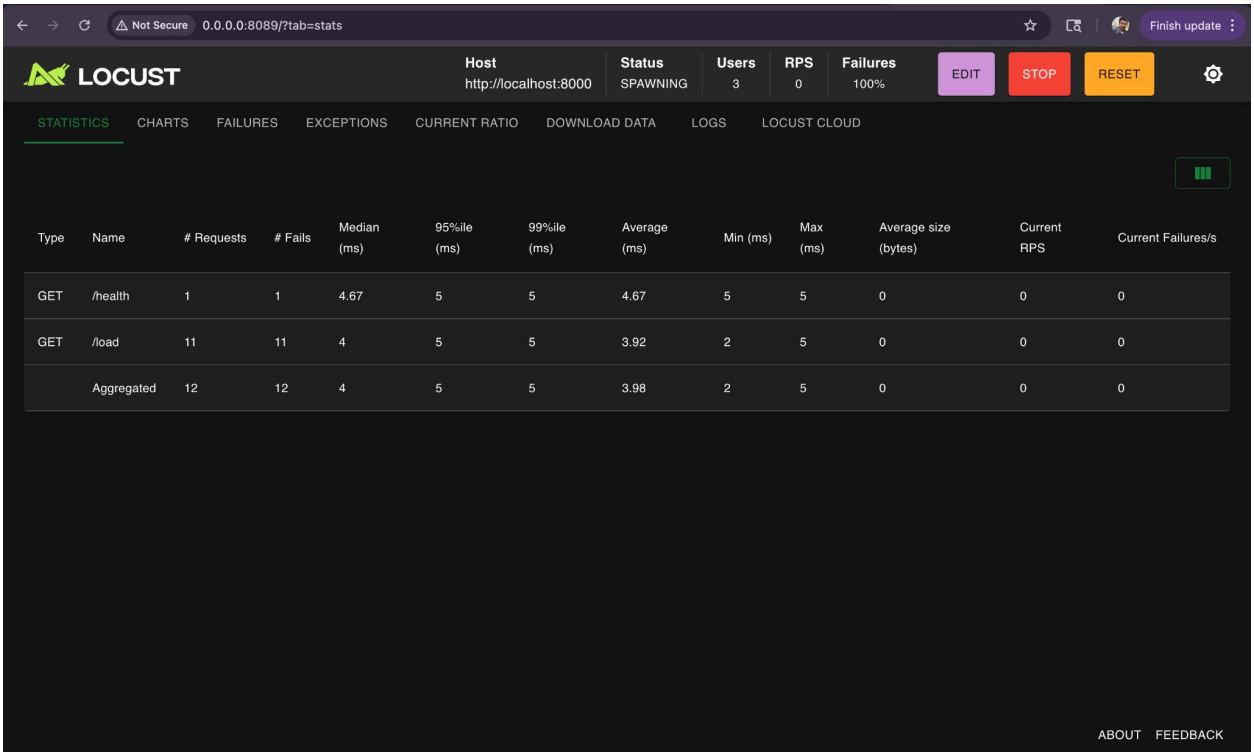


Figure 6.2: Locust Statics View

6.3 Analysis of the Results

This section explains how the observed experimental outcomes verify the selected TRs. Screenshots from the kubectl output and the Locust UI provide concrete evidence.

6.3.1 TR7.1 – Metric-driven scaling of compute capacity

TR7.1 requires the system to adjust compute capacity based on workload, scaling up when the system is busy and scaling down when demand subsides.

During the autoscaling phase, kubectl get hpa payment-hpa -w (Figure 6.3) shows CPU utilization rising from 1–3% at idle to approximately 289%, well above the target of 50%. As soon as CPU stays above the target, the HPA increases the REPLICAS value:

- Initially REPLICAS = 1
- Then REPLICAS is raised to 4
- Finally REPLICAS reaches 6

The pod watch output (Figure 6.4) confirms that new pods are actually created: pods appear in Pending, transition to ContainerCreating, and quickly reach Running state. After a short

stabilization period, kubectl get pods -l app=payment (Figure 6.5) shows six healthy pods, all READY 1/1.

This behavior is entirely driven by metrics: no manual kubectl scale operations were performed. CPU metrics collected by Kubernetes triggered the autoscaler, which updated the Deployment's spec.replicas, exactly matching the intent of TR7.1. The results demonstrate that the system can elastically increase capacity when load increases.

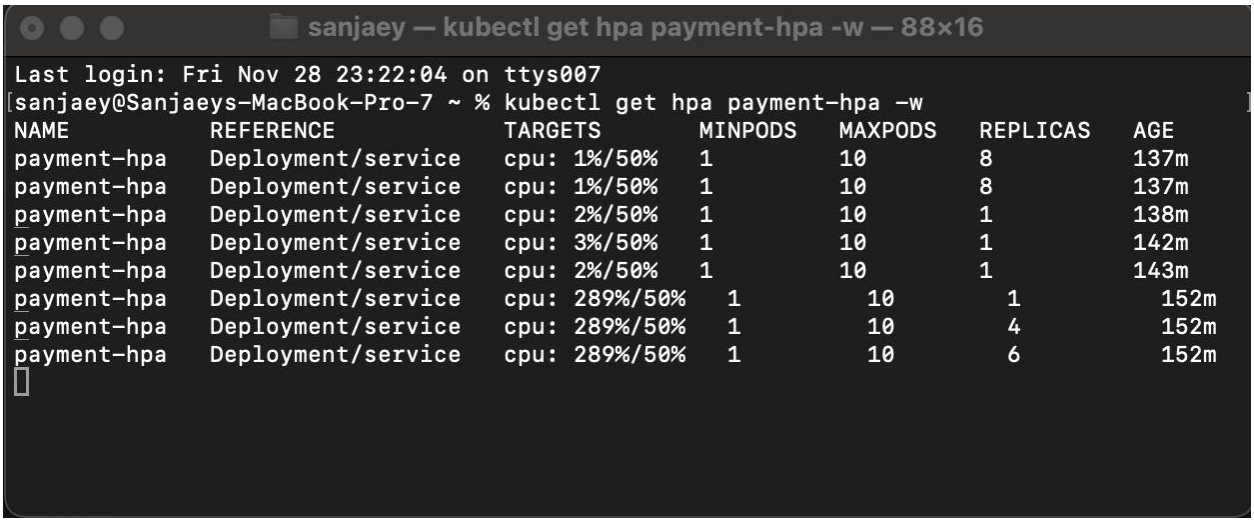


Figure 6.3: Output of kubectl get hpa payment-hpa -w, illustrating CPU utilization

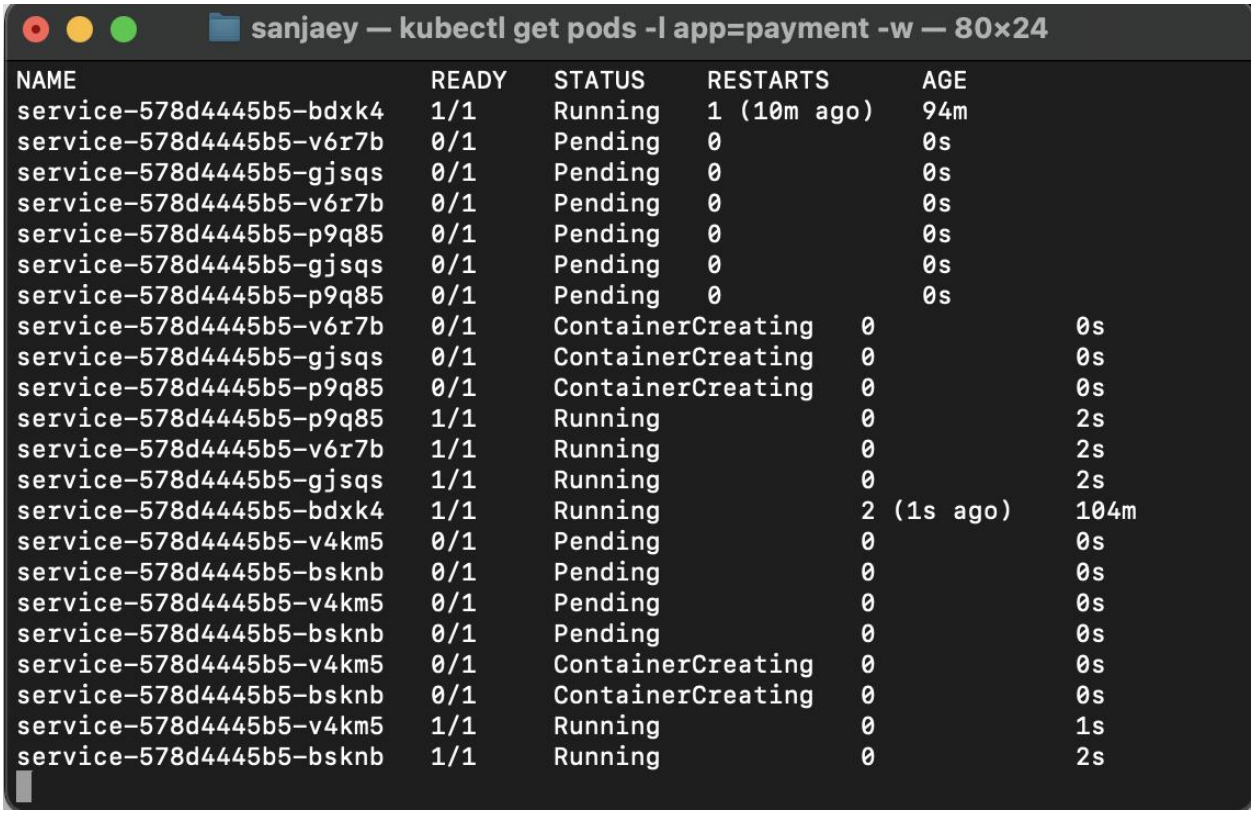


Figure 6.4: Output of kubectl get pods -l app=payment -w during scale-out, showing new pods appearing

```
sanjaey@Sanjaeys-MacBook-Pro-7 ~ % kubectl get pods -l app=payment
NAME                       READY   STATUS    RESTARTS       AGE
service-578d4445b5-bdxk4   1/1     Running   2 (100s ago)   105m
service-578d4445b5-bsknb   1/1     Running   0              92s
service-578d4445b5-gjsqs   1/1     Running   0              107s
service-578d4445b5-p9q85   1/1     Running   0              107s
service-578d4445b5-v4km5   1/1     Running   0              92s
service-578d4445b5-v6r7b   1/1     Running   0              107s
sanjaey@Sanjaeys-MacBook-Pro-7 ~ %
```

Figure 6.5: Steady-state view of kubectl get pods -l app=payment, with six service-pods all in Running 1/1 status, representing the stable scaled-out configuration under load.

6.3.2 TR7.2 – Parallel handling of multiple jobs

TR7.2 requires that multiple jobs or requests can run in parallel instead of queuing behind a single worker.

Once the Deployment scaled out to six replicas (Figure 6.5), Locust continued to send requests from ten simulated users. Because the NodePort Service load-balances across all app=payment pods, concurrent /load calls were processed by multiple pods at the same time. The Locust statistics (Figure 6.2) show steady throughput with low median and 95th-percentile latencies, indicating that the additional replicas are actually being used and that traffic is not serialized through a single bottleneck.

This confirms TR7.2: the system can exploit horizontal scaling to process many independent requests in parallel, which maps directly to running multiple preprocessing jobs concurrently in the real pipeline.

6.3.3 TR2.1 and TR2.4 – Redundancy and self-healing for availability (Verified)

TR2.1 requires redundancy for critical compute components so that a single failure does not cause downtime. TR2.4 requires continuous health checks and mechanisms to remediate failures automatically.

To test these properties, we performed a controlled failure: after the cluster reached a stable configuration with six pods, we deleted one pod manually using kubectl delete pod ... (Figure 6.6). At the same time, we kept kubectl get pods -l app=payment -w running.

Immediately after deletion, the affected pod moved to the Terminating state. Almost at once, a new pod appeared in Pending and transitioned to ContainerCreating and then Running (Figure 6.7). Throughout this sequence, the other pods remained Running 1/1, so there was no loss of capacity from the Service's perspective traffic continued to flow to the remaining pods while the replacement came up.

Although this test used a manual delete, it mimics what would happen if a pod crashed or failed its liveness probe. Because the Deployment is configured with a liveness check on /health, Kubernetes continuously monitors pod health and would automatically restart or replace any pod that stops responding, even without explicit user intervention. This behavior directly satisfies **TR2.4**, since Kubernetes continuously checks pod health and automatically replaces unhealthy replicas.

This confirms both TR2.1 and TR2.4:

- **TR2.1 (Redundancy)** – Losing a single pod did not cause downtime; the remaining replicas continued to serve traffic while a replacement was created.

- **TR2.4 (Health checks)** – The combination of liveness probes and the Deployment's reconciliation loop ensures that unhealthy pods are detected and replaced, providing continuous self-healing.

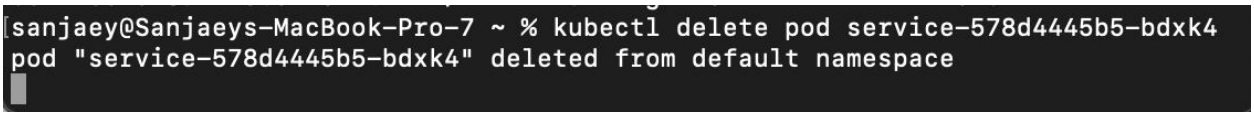


Figure 6.6: Command line output of kubectl delete pod service-578d4445b5-bdxk4, manually removing one running pod to test Kubernetes self-healing behavior.

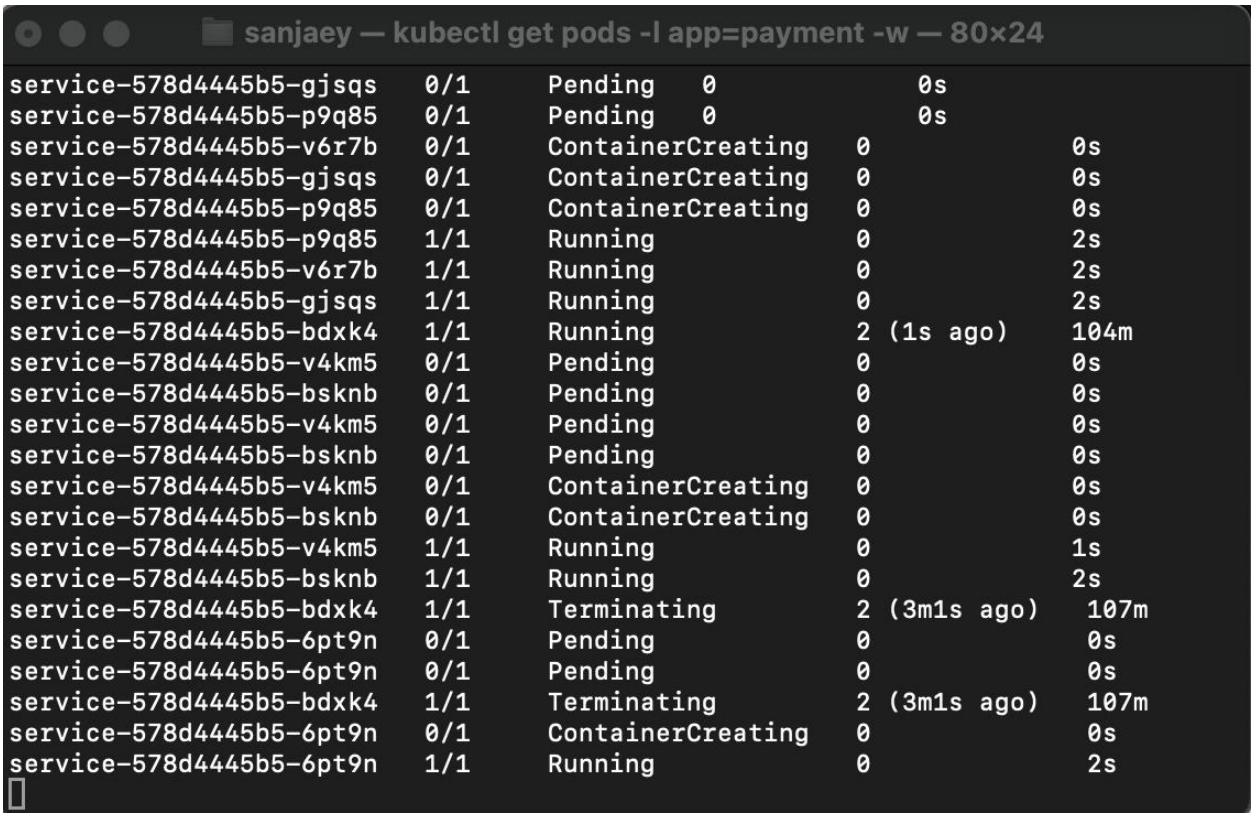


Figure 6.7: Output of kubectl get pods -l app=payment -w after the deletion, showing the deleted pod in Terminating state and a new replacement pod progressing to Running, while other pods remain healthy—demonstrating automatic self-healing and maintained availability.

6.3.4 TR4.1 – Metrics collection to support autoscaling and monitoring (Verified)

TR4.1 requires that the system record metrics such as runtimes, throughput, and failure rates to support monitoring and autoscaling decisions.

In this experiment, two different layers of metrics were visible:

- **Kubernetes/HPA metrics** – CPU utilization per pod was continuously measured and used by the HPA to adjust replica counts (Figure 6.3). This is the same category of metric that, in the full AWS design, would be exported to CloudWatch as autoscaling signals.

- **Application-level metrics from Locust** – Locust recorded total requests, success/failure counts, and detailed latency distributions for each endpoint (Figure 6.2). These metrics reflect the actual user experience of the preprocessing API.

By combining these metrics, we can reason both about system health (latency, failures) and about internal resource utilization (CPU, replicas). The experiment therefore demonstrates that our architecture exposes sufficient metrics to drive autoscaling and to detect performance problems, satisfying TR4.1. This confirms **TR4.1**, because the system exposes both infrastructure-level metrics (CPU) and application-level metrics (latency, failures) required for scaling and monitoring.

6.3.5 Overall Confirmation

The Kubernetes experiments collectively validate that the key reliability and scalability mechanisms in our design behave as required:

- **Elastic scaling (TR7.1)** – The HPA increased replicas from 1 to 6 when CPU utilization exceeded the 50% target, and maintained higher capacity while load persisted.
- **Parallel processing (TR7.2)** – Multiple pods processed concurrent requests from multiple simulated users, preventing contention on a single worker.
- **Availability and self-healing (TR2.1, TR2.4)** – Deleting a pod did not disturb the service: Kubernetes immediately created a replacement pod and ensured that other replicas remained healthy.
- **Metrics-driven control (TR4.1)** – Both infrastructure-level and application-level metrics were available and directly influenced autoscaling behavior.

Although these experiments were run on a small local Minikube cluster with a synthetic Flask service, the behavior transfers directly to the AWS-based preprocessing platform. In the final design, containerized preprocessing workers will be deployed behind a service, scaled by workload metrics, and monitored through health checks, just as in this Kubernetes validation. The results here provide concrete experimental evidence that the design decisions in Sections 4 and 5 are feasible and satisfy the selected TRs.

7. Ansible Playbooks

Skipped as per project guidelines.

8. Demonstration

Skipped as per project guidelines

9. Comparison: CloudFormation vs Terraform

For this section, I chose to compare two Infrastructure-as-Code (IaC) tools: **AWS CloudFormation** and **HashiCorp Terraform**. Both tools automate the provisioning of cloud resources, but they differ in scope, flexibility, and workflow. Since CloudFormation is used in our project, this comparison helped me understand why it is suitable for AWS-centric architectures and where Terraform would offer advantages.

9.1 Overview

CloudFormation is AWS's native IaC service. It manages AWS resources through JSON/YAML templates and integrates tightly with IAM, CloudWatch, CloudTrail, and service-specific features. It is optimized for single-cloud AWS deployments.

Terraform is a multi-cloud IaC tool. It uses its own declarative language (HCL) and can provision resources across AWS, GCP, Azure, and many other providers. Terraform also supports more complex inter-cloud configurations and state management through an external backend.

9.2 Comparison

1. Cloud Support

- **CloudFormation:** Only supports AWS.
- **Terraform:** Supports AWS, Azure, GCP, Kubernetes, SaaS services, and on-prem systems.

Implication for our project:
Since our main architecture lives entirely on AWS, CloudFormation fits cleanly. Terraform would only help if we wanted to provision shared infrastructure across AWS + GCP.

2. Syntax and Language

- **CloudFormation:** YAML/JSON. Verbose but predictable, directly maps to AWS APIs.
- **Terraform:** HCL, more readable and expressive, supports modules and reusability better.

Implication:
Terraform would make templates easier to manage as the system grows, especially if different teams reuse modules.

3. State Management

- **CloudFormation:** Maintains stack state automatically inside AWS.
- **Terraform:** Requires an external state backend (e.g., S3 + DynamoDB lock table).

Implication:
CloudFormation simplifies operations because there is no separate state file to secure. Terraform's state system is more flexible but more operationally sensitive.

4. Change Detection and Updates

- **CloudFormation:** Provides "change sets" that show how updates modify a stack.
- **Terraform:** Provides a full "plan" showing what will be created, changed, or destroyed.

Implication:
Terraform plans are usually easier to read, but CloudFormation change sets integrate directly with AWS's event logs.

5. Multi-Cloud and Hybrid Use Cases

- **CloudFormation:** No first-class support.
- **Terraform:** Designed for hybrid and multi-cloud deployments.

Implication for our hybrid AWS → GCP architecture:

Terraform could help define both sides (AWS + GCP) in one place. However, GCP only consumes processed data in our design, so we don't need provisioning on the GCP side.

6. Ecosystem and Tooling

- **CloudFormation:** Strong AWS support, SAM/CDK integrations, StackSets for multi-account management.
- **Terraform:** Very large ecosystem with reusable modules (Terraform Registry).

Implication:
CloudFormation keeps everything inside AWS without extra tooling, which matches our single-cloud deployment model.

Category	CloudFormation	Terraform
Cloud Support	AWS only	Multi-cloud (AWS, GCP, Azure, Kubernetes, etc.)
Language	YAML/JSON	HCL (more compact and modular)
State Management	Built-in AWS-managed stack state	External state backend required (S3, GCS, etc.)
Reusability	Limited module system; relies on nested stacks	Strong module ecosystem (Terraform Registry)

Change Preview	Change Sets	Terraform Plan (more readable)
Ecosystem Integration	Deeply integrated with IAM, CloudWatch, CloudTrail	Broad ecosystem, supports many providers
Learning Curve	Easier for AWS-only teams	Slightly higher due to state & workflows
Multi-Cloud / Hybrid Fit	Weak (AWS-centric only)	Strong, designed for hybrid environments
Operational Overhead	Lower (no custom state files)	Higher (state management, locking, drift checks)
Best Use Case	Single-cloud AWS deployments	Multi-cloud provisioning and reusable IaC modules

9.3 Conclusion

For a strictly AWS-based workload like the preprocessing platform, **CloudFormation** is the simpler and more natural choice. It integrates directly with IAM, CloudWatch, CloudTrail, and the AWS console, which reduces operational overhead and keeps the environment cohesive.

Terraform offers more flexibility, better modularity, and multi-cloud capabilities. If the project required provisioning infrastructure on both AWS and GCP or if the company later expanded to a hybrid or multi-provider model, Terraform would become more useful. For the current architecture, however, CloudFormation aligns better with the design goals and keeps the deployment model straightforward.

10. Conclusion

10.1 Lessons Learned

Working on this project was a lot more involved than I expected, and I ended up learning much more about cloud architecture than I thought I would. The biggest challenge for me was understanding how Business Requirements and Technical Requirements actually shape the design. At the start, I treated them like formalities, but once we began connecting them to the Well-Architected Framework, I realized how much they drive the structure of the whole system.

Another part that was more difficult than I expected was separating “design decisions” from “solutions.” I had to rewrite several sections because I kept slipping into implementation details too early. The process forced me to slow down and think in layers - identity, data, compute, observability - rather than jumping straight to services.

The most interesting part was learning how multi-tenancy, hybrid cloud, and monitoring all connect. I’ve used AWS before, but I never had to think about tenant isolation, cross-cloud security boundaries, or how metrics actually influence scaling decisions. This project made me see how these pieces fit together and how much planning goes into making a cloud system reliable.

Creating the architecture diagram was surprisingly fun. Laying out the data plane and control plane made the entire design feel more real. It also helped me catch things I missed earlier, like where logging needed to be captured or how tenants actually authenticate into the system.

Overall, the process was more structured and rigorous than I expected, but it gave me a much clearer picture of how a real cloud architecture is built and validated.

10.2 Possible Continuation of the Project

Even though I’m not planning to do an Independent Study or take an advanced cloud course next semester, there are several natural directions this project could grow into.

One idea is to extend the system into a full multi-account setup using AWS Organizations. That would make tenant isolation even stronger and would allow separate environments for development, testing, and production. Another direction is to add automated integration tests that validate tenant boundaries, data versioning, and hybrid cloud transfers. These tests could run as part of a CI/CD pipeline.

A more advanced extension would be to explore cross-cloud orchestration - either using Terraform to provision both AWS and GCP infrastructure in one place, or experimenting with event-driven transfers from S3 to GCP. This wasn't necessary for the current project, but it would be interesting to see how both environments can be treated as parts of a single workflow.

Another possible extension is to simulate real preprocessing loads with synthetic time-series data and measure how far the architecture scales before hitting bottlenecks. This could lead to more detailed performance tuning and even more advanced autoscaling strategies.

Even though I won't continue this in a formal course, these are areas that I think would be interesting to explore if I ever revisit the design.

11. References

Below is the format your professor expects. I've added placeholders for any external references (AWS docs, HashiCorp docs, RFCs, etc.). You can add more as needed.

1. Y. Viniotis and Y. Pan, *ECE547/CSC547 Class Notes*, 2023.

2. AWS Identity and Access Management Documentation: https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

3. Amazon S3 Documentation: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

4. Amazon CloudWatch Documentation: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

5. AWS CloudTrail Documentation: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html

6. Amazon ECS Documentation: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

7. HashiCorp Terraform Documentation: https://developer.hashicorp.com/terraform/docs